# Parallel Computing, 2026s

**Theory II: Parallelism Fundamentals**

University of Vienna

# Outline

- Concurrency vs. Parallelism
- Speedup and Parallel Efficiency
- Amdahl's Law
- Classification of Parallel Architectures
- Data Locality
- Parallel Programming Models
- Data Parallelism vs. Task Parallelism
- Dependences and Data Races
- Load Balancing and Granularity
- Weak and Strong Scalability
- Peak Performance, Sustained Performance, Efficiency

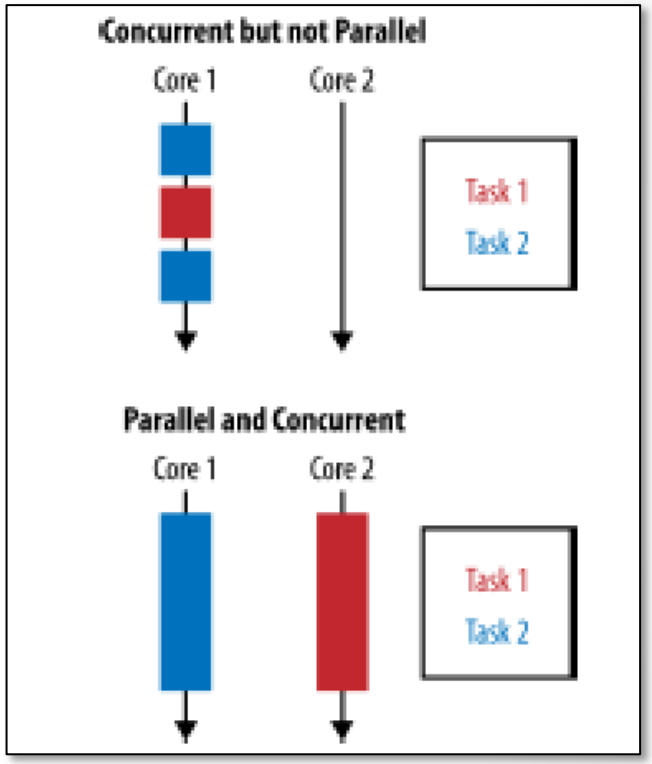
# CONCURRENCY VS. PARALLELISM

## Concurrency

- A condition of a system in which multiple tasks are **logically** active at one time
- * tasks may or may not execute at the same time

## Parallelism

- A condition of a system in which multiple tasks are **actually** active at one time




See: Java 8 Lamdas.

# WRITING A PARALLEL PROGRAM

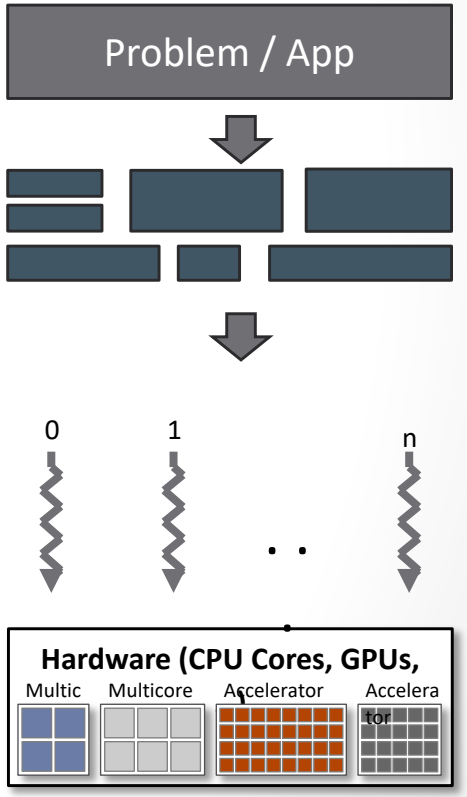
**Typical goal of parallelism**

- To run programs faster by using multiple "processors"

$$S_p = \frac{time\ on\ 1\ processor}{time\ on\ P\ processors}$$

**How?**

- **Decomposition** of a computational problem into subproblems that can be executed in parallel
  - Partition work and data
  - Identify dependences
- **Assign** (allocate) work (or tasks) to available "workers" (e.g., threads)
  - Goals: workload balance, reduction of communication costs
  - Can be static and dynamic
- **Manage** data access, **communication** and **synchronization**
- **Mapping** to hardware
  - Workers to execution units (e.g., threads)
  - OS vs Compilers
  - Mapping policies



# CONCURRENCY VS. PARALLELISM

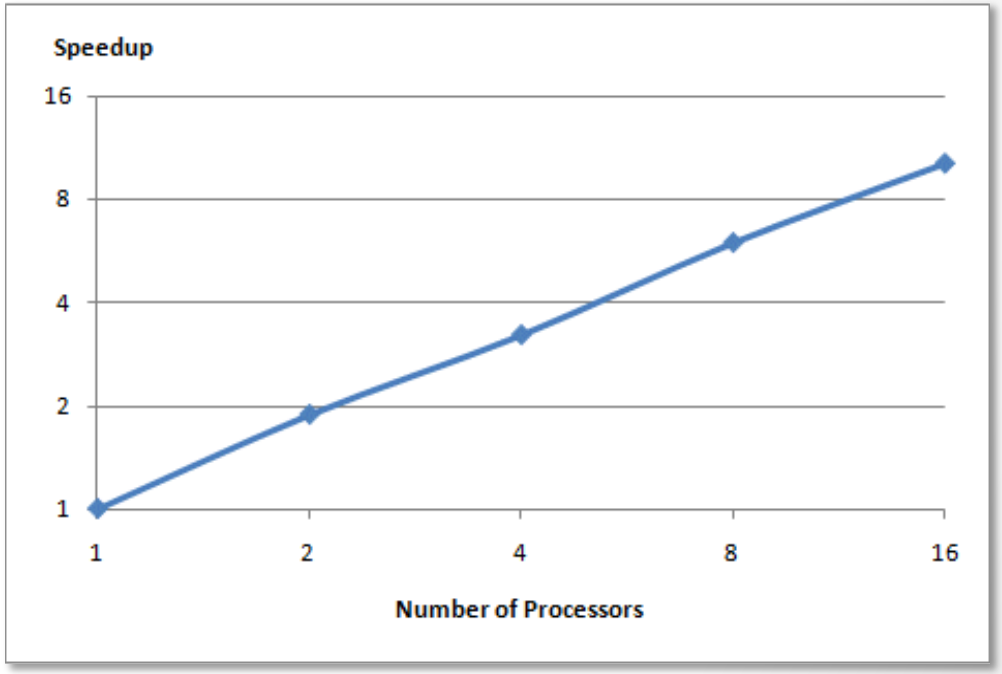
## Goal of Concurrency

- To effectively manage and coordinate multiple tasks

## Goal of Parallelism

- To run programs faster by using multiple processors

- **Speedup**: $S(p) = T_s / T(p)$
  - $T_s$ ... sequential execution time
  - **T(p)** ... execution time on **p** processors



# BASIC PERFORMANCE MEASUREMENTS

**Execution time, response time, wall-clock time, elapsed time:**

→ Time between the start and the completion of an event

**Parallel Speedup on P processors: $S(p) = T_s / T_p$**

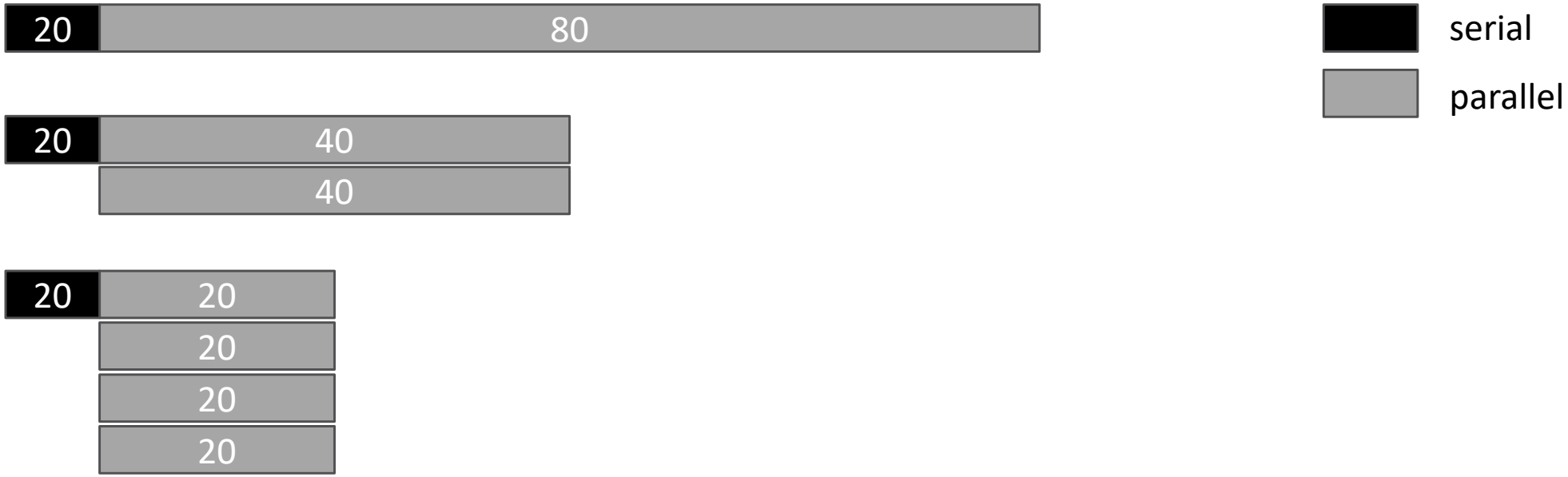
- $T_s$ ... execution time of sequential program
- $T_P$ ... execution time of parallel program on **p** processors
- Remarks:
  - Sometimes speedup measured as $T_1 / T_P$
  - **S = P** → linear speedup (rare, e.g., embarrassingly parallel programs)
  - $T_p$ is a wall-clock time, hard to measure

**Parallel Efficiency on P processors: $E(p) = S(p)/p$**

- Range [0..1]
- E = 1 (100%, linear speedup) → very rare due to overheads because of communication, contention, load imbalance

# PERFORMANCE :: OBSERVATIONS

**Serial and parallel fractions**




…

# AMDAHL'S LAW

*Performance improvement to be gained from using some faster mode of execution is limited by the fraction of time the faster mode can be used.*

**Parallel execution time on N processors:**

$$T_p = T_s\left(\left(1 - F_p\right) + \frac{F_p}{S_p}\right)$$

**Overall speedup on N processors:**

$$S_{overall} = \frac{T_s}{T_p} = \frac{1}{\left(1 - F_p\right) + \frac{F_p}{S_p}}$$

**Maximum possible speedup (N → ∞):**

$$S_{max} = \frac{1}{F_s}$$

**Where**

- $F_S$ - serial fraction of code,
- $T_s$ - sequential execution time
- $S_p$ - speedup of parallel fraction on N processors

$F_p$ = parallel fraction of code

$T_p$ = parallel execution time on N processors

# AMDAHL'S LAW

**Example:**

40% of program (execution time) can be parallelized with speedup 10

60% of program (execution time) is in sequential code and cannot be improved

$$\text{Speedup}_{\text{overall}} = \frac{1}{(1 - F_p) + \frac{F_p}{S_p}}$$

$$= \frac{1}{(1 - 0.4) + \frac{0.4}{10}} = \frac{1}{0.64} = 1.56$$

**Example:**

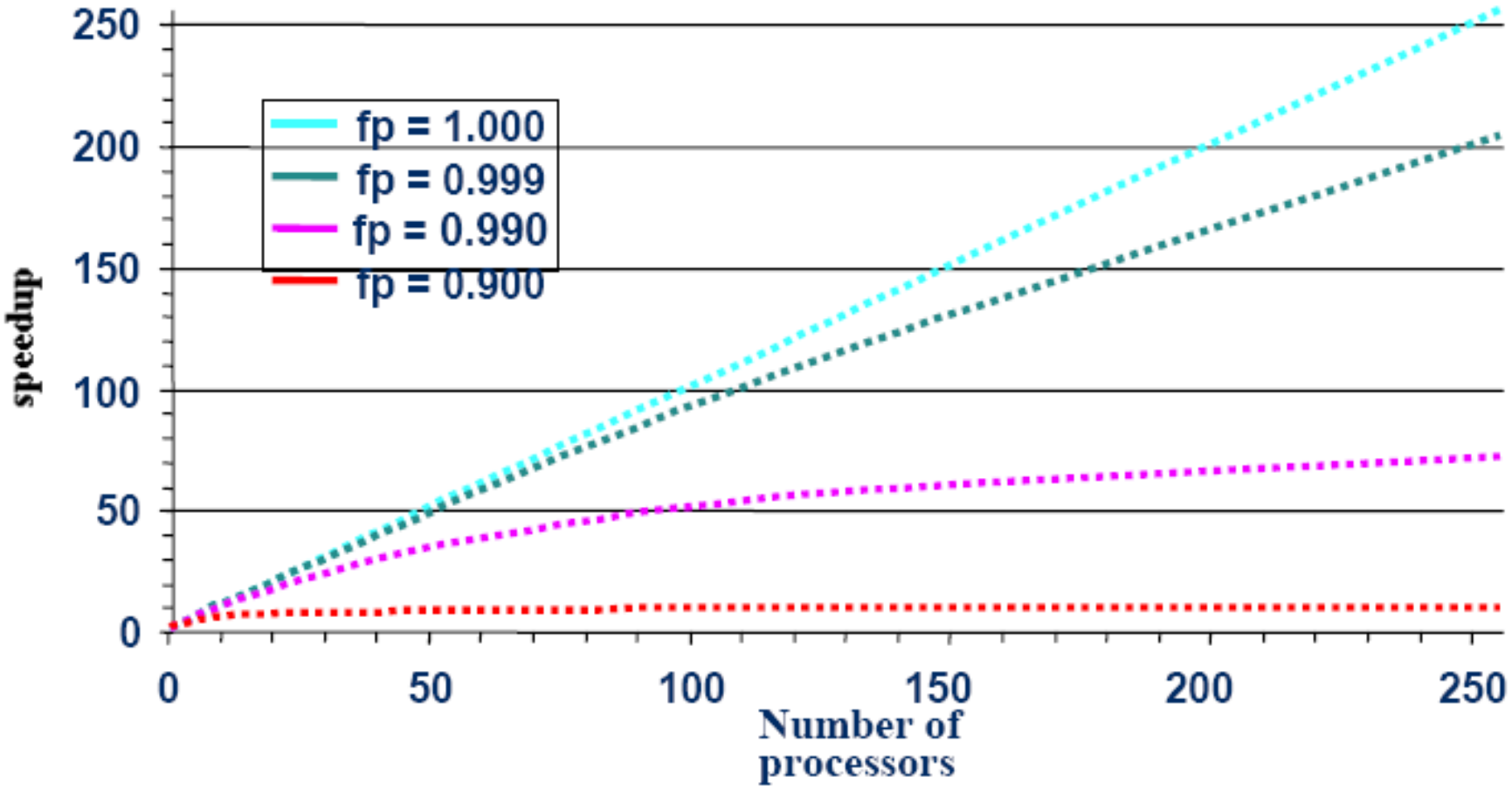
80% of program can be parallelized

20% of program is sequential

→ Max. speedup of 5 regardless of how many processors are used!!

# AMDAHL'S LAW

**Parallel speedup is limited by the serial fraction of the code.**



# PARALLEL ARCHITECTURES :: CLASSIFICATION

## Flynn's Classification

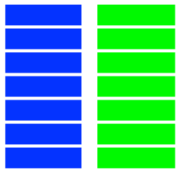
- Along two dimensions *Instruction Stream* and *Data Stream*
- **SIMD** – Single Instruction Multiple Data
  - E.g., GPU Streaming Multiprocessors
- **MIMD** – Multiple Instruction Multiple Data
  - e.g., multicore CPUs
- **SISD** – Single Instruction Singe Data
  - E.g., simple sequential processor
- **MISD** – Multiple Instructions Single Data
  - E.g., fault tolerant computing

## Classification wrt. Memory/Address Space

- Shared Memory vs. Distributed Memory
- Shared Address Space vs. Distributed Address Space

# FLYNN'S CLASSIFICATION

*Single Instruction, Single Data (SISD)*



e.g., sequential CPUs

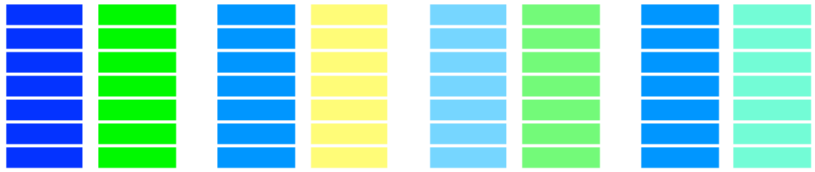
*Single Instruction, Multiple Data (SIMD)*

e.g., vector units, GPUs

*Multiple Instruction, Single Data (MISD)*

e.g., fault-tolrant controllers

*Multiple Instruction, Multiple Data (MIMD)*



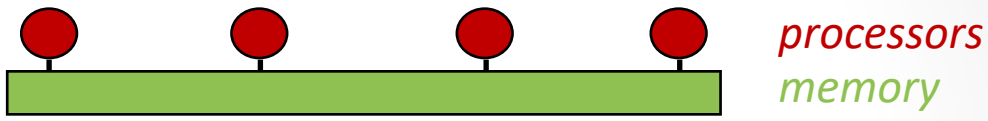
e.g., modern mulit-core CPUs

by Michael Flynn, proposed in 1966 and extended in 1972

# PARALLEL COMPUTER ARCHITECTURES

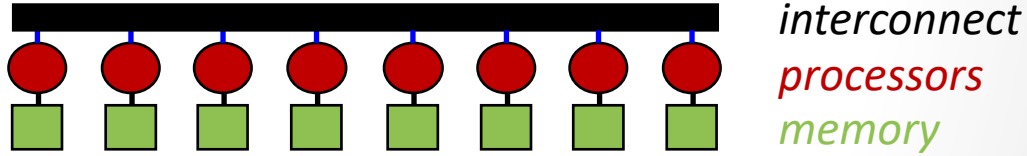
## Shared-memory Machines

- Not scalable
- Shared address space → easier to program
  - All processors can access memory




## Distributed-memory Machines

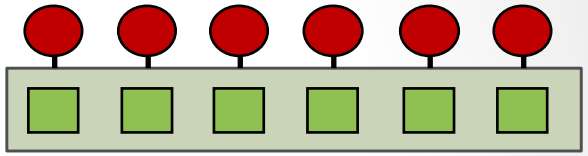
- Scalable to large number of processors
- Distributed address space → hard to program



# PARALLEL COMPUTER ARCHITECTURES

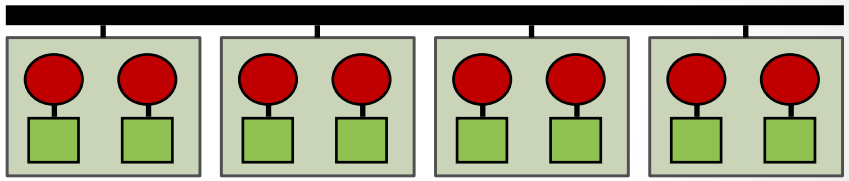
## Scalable (distributed) shared-memory machines

- Provide shared address space
  on top of distributed-memory hardware (NUMA)
- Limited scalability



## Clusters

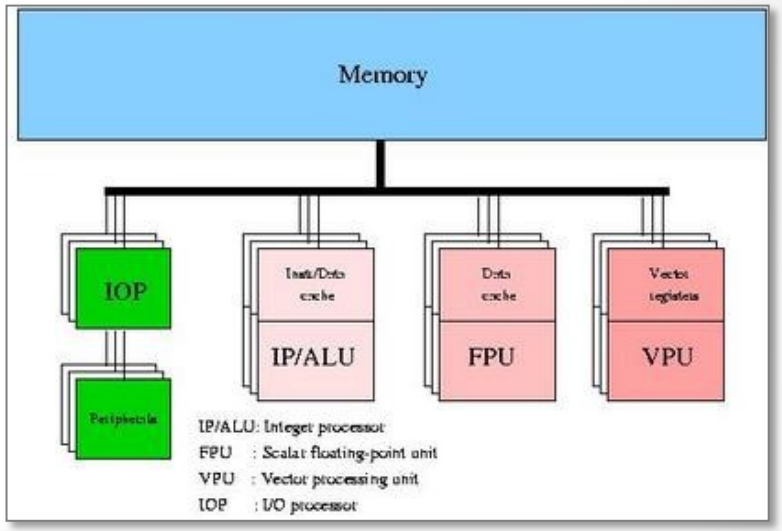
- Distributed-memory across nodes
- Shared-memory within nodes
- Scalable to large number of nodes
- **No globally shared address space**



# PARALLEL COMPUTER ARCHITECTURES

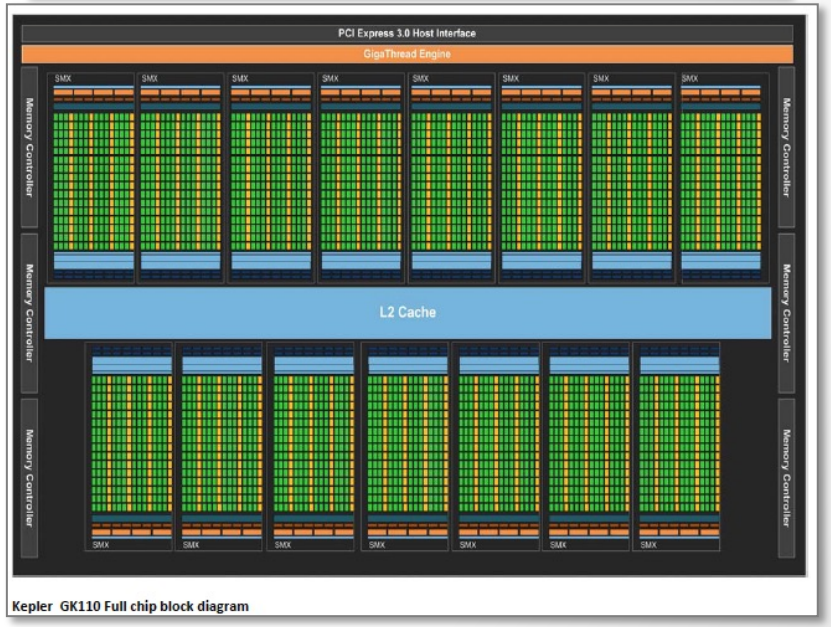
## Vector Machines

- Vector registers operate on vectors of data with 64-1024 elements (64 bit)
- Shared-memory SIMD
- Examples: Cray-1, NEC SX9




## Graphic Processing Units (GPUs)

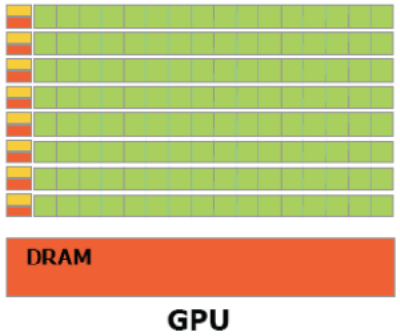
- Organized as a set of SIMD-like processors e.g. NVIDIA Streaming Multiprocessors (SM, SMX)
- Each SM comprises many simple cores
- Fine-grain, lightweight threads (>>1000)
- Low single-thread performance



# GPUs vs CPUs

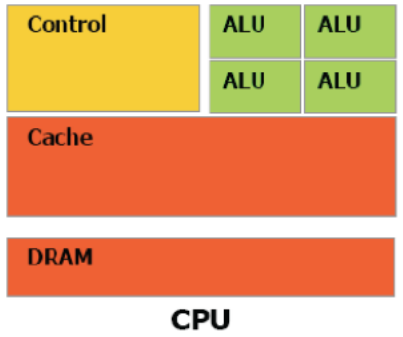
## GPUS very efficient for

- High Computation per Memory Access
- Single Instruction Multiple Data Operations
- Fast Parallel Floating-point Processing

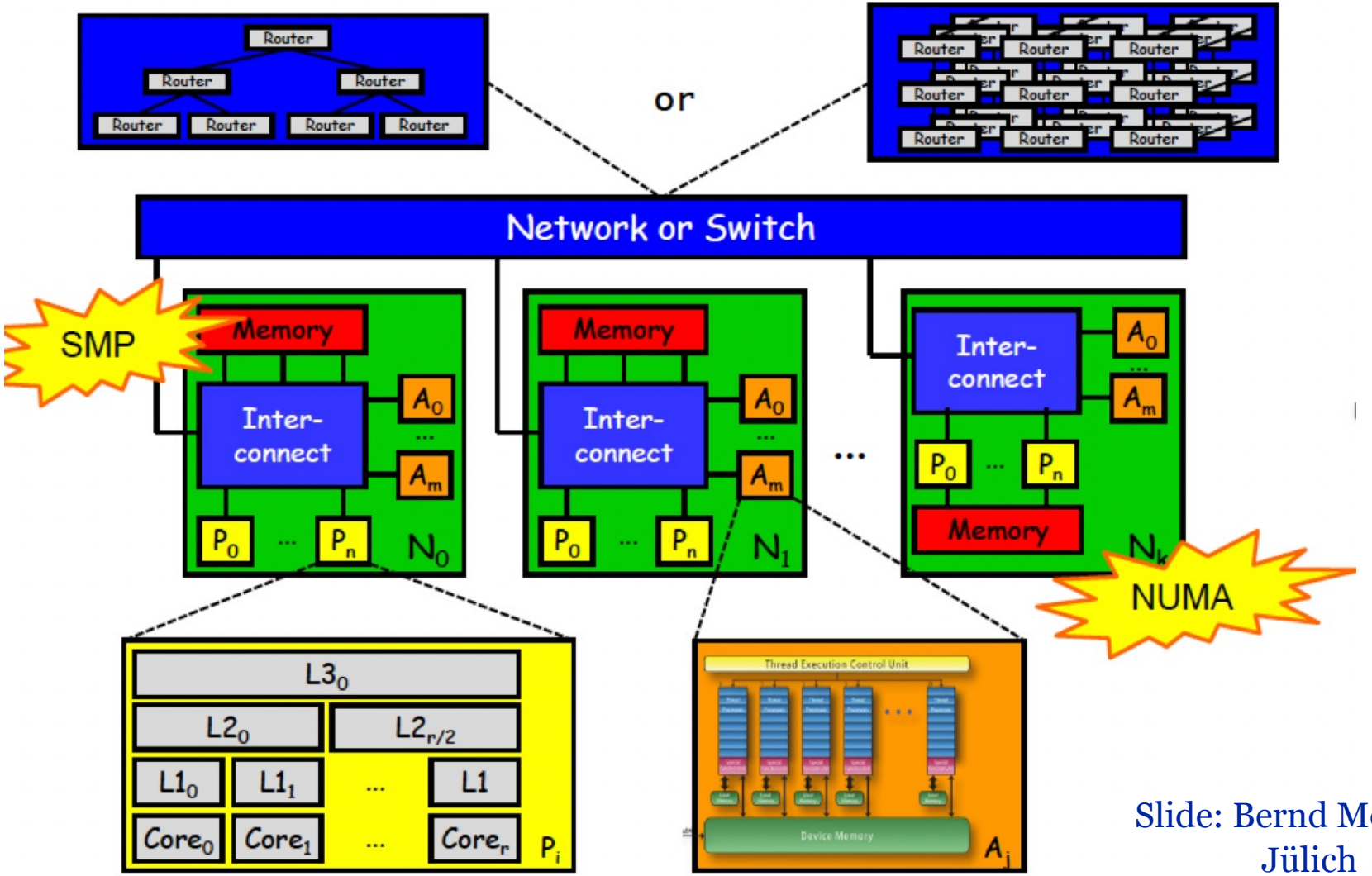



## GPUs not as efficient for

- Random Access, Memory-Intensive Operations
- Logical Operations on Integer Data
- Branching-Intensive Operations



# CURRENT HPC ARCHITECTURES (CLUSTERS)




Slide: Bernd Mohr, FZ Jülich

# LEVELS OF PARALLELISM

## Core-Level Parallelism

- Instruction Level Parallelism (ILP)
  Pipelined instruction execution; superscalar execution; out-of-order execution
- SIMD Parallelism (within a core)
  Multimedia Extensions (MMX), SSE (Streaming SIMD Extensions), AVX (Advanced Vector Extensions)

## Chip-Level Parallelism (shared memory)

- Multiple cores per processor chip

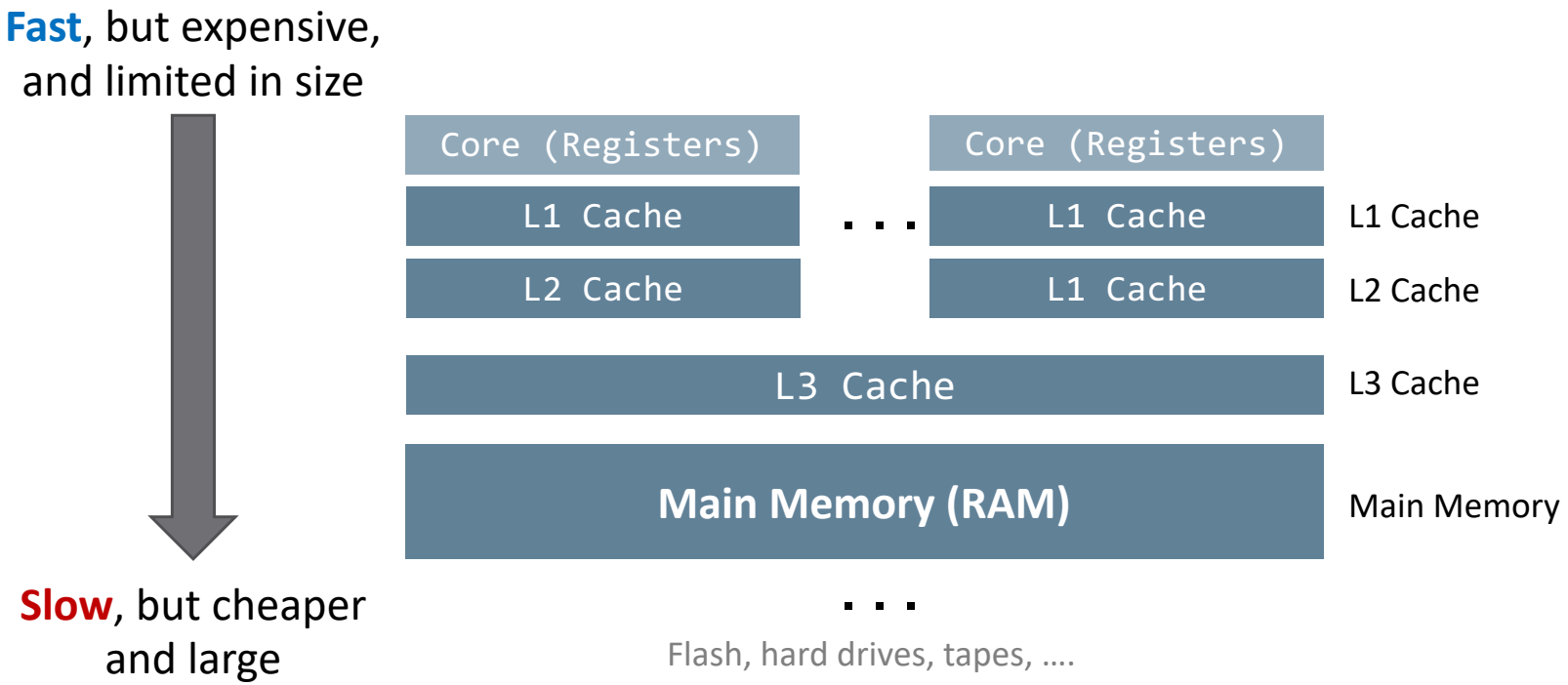
## Node-Level Parallelism (shared memory/distributed memory)

- Multiple sockets (multicore processor chips) per node (cf. SMPs)
- Accelerators/GPUs (connected via PCIe)

## Cluster-Level Parallelism (distributed memory)

- Multiple nodes connected via network (distributed-memory)

# MEMORY HIERARCHY (MULTI-CORE CPU)




**Memory hierarchies exploit**

- temporal locality by keeping more recently accessed data items closer to processor
- spatial locality by moving blocks in memory to upper level of hierarchy

***Hit*** - data requested by the processor is in the upper level or
***Miss*** – if not found in the upper level, usually incurring a ***miss penalty***
*The smallest piece of information that is transferred is either a block or a line

# DATA LOCALITY

## Principle of Locality

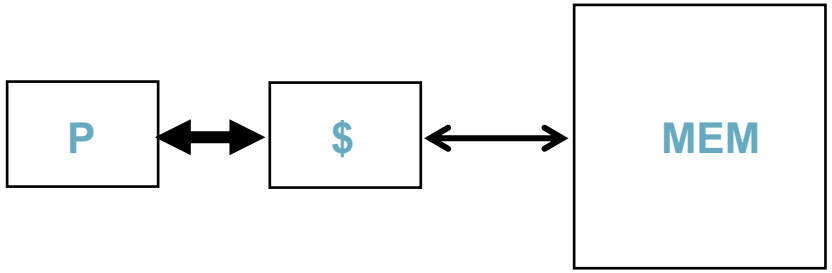
- Programs access a relatively small portion of the address space at any instant of time.

## Temporal Locality (Locality in Time)

- If an item is referenced, it will tend to be referenced again soon (e.g., loops).

## Spatial Locality (Locality in Space)

- If an item is referenced, items whose addresses are close by tend to be referenced soon (e.g., straight-line code, array access)



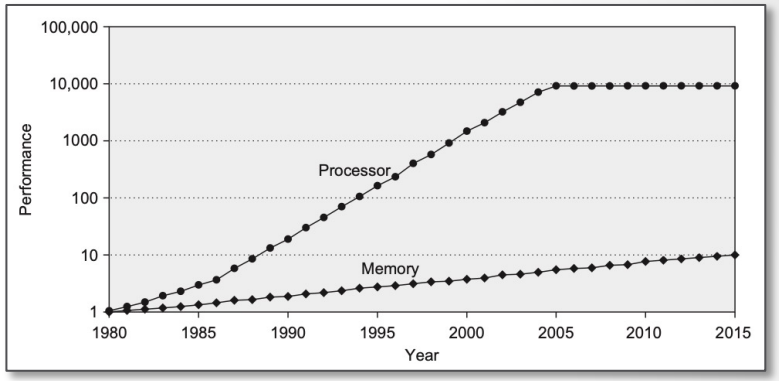

**Data locality is key to performance!**

# MEMORY WALL

**Processor-Memory performance gap**

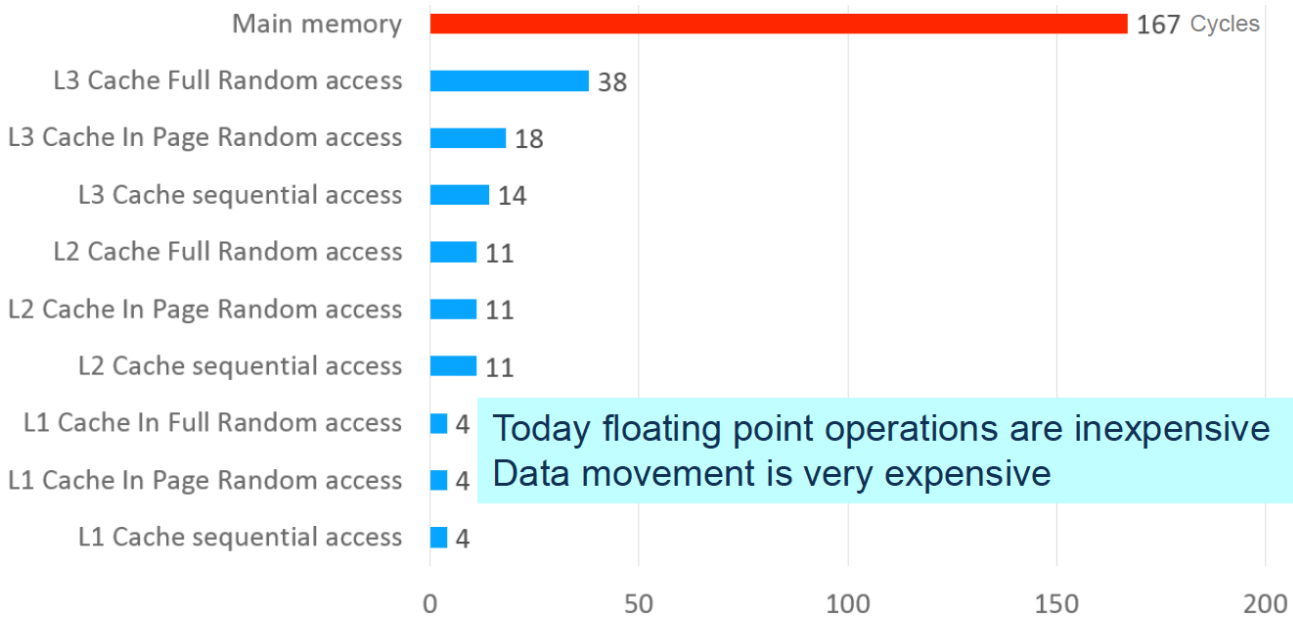
**Relative increase of memory latency**

- Deep memory hierarchies (caches)




Graph source: Computer Architecture:
A Quantitative approach,
Patterson, Hennessy

**Example: Intel Xeon Skylake (AVX 512) - CPU Access Latencies in Cycles**




Today floating point operations are inexpensive
Data movement is very expensive

**32** FLOPS/cycle/core (**DP**) → **167** cycles = **5344** FLOPS (DP)

Source: Jack Dongarra

# PARALLEL PROGRAMMING MODELS

## Parallel programming models differ in:

- How parallelism is organized
  - *Data parallel models* vs. *Task parallel models*

and

- How memory is organized
  - Shared memory (Java/C++) vs. Distributed memory (MPI)

## Parallel programming involves (among other details)

- distributing work to threads/processors
- distributing data (if memory is distributed)

and

- synchronization (e.g. of accesses to shared data)
- communication of data between processors (if memory is distributed)

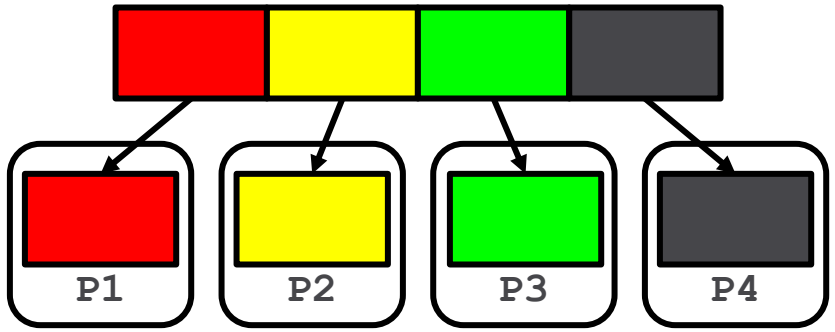
# DATA PARALLELISM

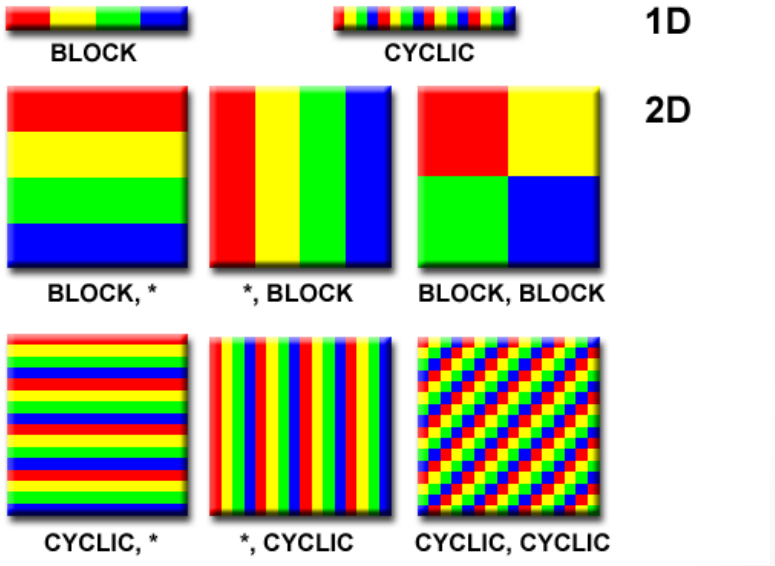
A single program is executed in parallel multiple times, each instance working on a different portion of the data.

Based on domain decomposition (cf. data partitioning/distribution)

SPMD Model: Single-Program-Multiple-Data

Generated by user/compiler and managed by compiler/runtime/hardware



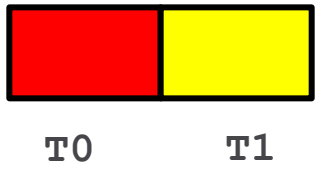



# DATA PARALLELISM :: SHARED MEMORY

**Sequential code**

```
double a[100];

for(int i=0; i<100; i++) {
   a[i]= i;
}
```




**Data in shared memory**

```
double a[100];
```

**Code executed by processing unit 0**

```
...
for(int i=0; i<50; i++) {
   a[i]= i;
}
```

**Code executed by processing unit 1**

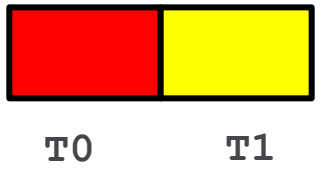
```
...
for(int i=50; i<100; i++) {
   a[i]= i;
}
```

# DATA PARALLELISM :: SHARED MEMORY

**Sequential code**

```
double a[100];

#pragma omp parallel for
for(int i=0; i<100; i++) {
   a[i]= i;
}
```




**Data in shared memory**

```
double a[100];
```

**Code executed by processing unit 0**

```
...
for(int i=0; i<50; i++) {
   a[i]= i;
}
```

**Code executed by processing unit 1**

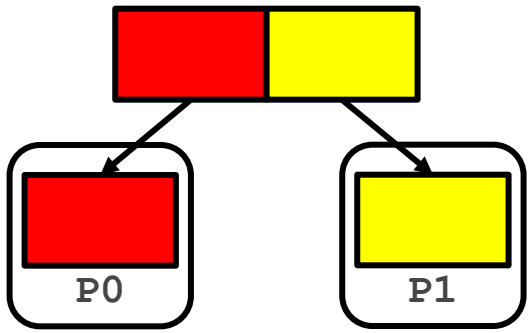
```
...
for(int i=50; i<100; i++) {
   a[i]= i;
}
```

# DATA PARALLELISM :: DISTRIBUTED MEMORY

**Sequential code**

```
double a[100];

for(int i=0; i<100; i++) {
   a[i]= i;
}
```




**Code executed by processing unit 0**

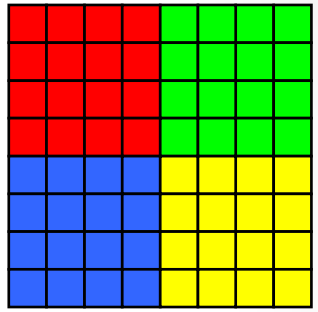
```
double a[50];
...
NP = 2;  // # processors
PID = 0; // id of processor
...
for(int i=0; i<50; i++) {
   a[i]= i + PID*50;
}
```

**Code executed by processing unit 1**

```
double a[50];
...
NP = 2;  // # processors
PID = 1; // id of processor
...
for(int i=0; i<50; i++) {
   a[i]= i + PID*50;
}
```

# DATA PARALLEL MODEL :: SPMD

## Single-Program Multiple-Data (SPMD)



- Each processor executes the same code
- Code is parameterized with processor/thread ID such that each processor/thread executes code on **different** data
- Communication (distributed memory) / Synchronization (shared memory) (if data from other processors/threads required/accessed)
- Scalar variables are typically replicated on all processors/threads and redundantly computed on each of them.

## Not to be confused with:

- SIMD: Single-Instruction-Multiple-Data (vectorization)
- SIMT: Single-Instruction-Multiple-Threads (e.g., CUDA, OpenCL)

# EXAMPLE: DATA & WORK DISTRIBUTION

## Data distribution

- Each processor gets part of data (local data)
- Work for local part of the data usually done by local processor (owner computes rule)

```
Global array: a(100)

→ P1: a(0:24)
→ P2: a(25:49)
→ P3: a(50:74)
→ P4: a(75:99)
```

## Work distribution

- Usually based on loop decomposition
- Dependences may inhibit parallelization

```
for (i=0;i<100;i++)

P1: i:  0..24
P2: i: 25..49
P3: i: 50..74
P4: i: 75..99
```

# COMMUNICATION/SYNCHRONIZATION :: EXAMPLE

**Assume following data distribution:**

```
P1: a(0:24)   b(0:24)  c(0:24)
P2: a(25:49)  b(25:49) c(25:49)
P3: a(50:74)  b(50:74) c(50:74)
P4: a(75:99)  b(75:99) c(75:99)
```

**Communication (in case of distributed memory)**

```
// sequential code
for(i=1;i<99; i++)
  c(i) = a(i) + f(b(i-1))
```

```
// first iteration on P2
c(25) = a(25)+f(b(24))
```

**Synchronization**

```
for(i=0;i<99; i++)            i=0..24|25..49|50..74|75..99
   b(i) = ...a(i)...
.......................................................<<barrier synchronization needed>>
for(i=0;i<=99; i++)           i=0..24|25..49|50..74|75..99
   c(i) = ...b(99-i)...
```

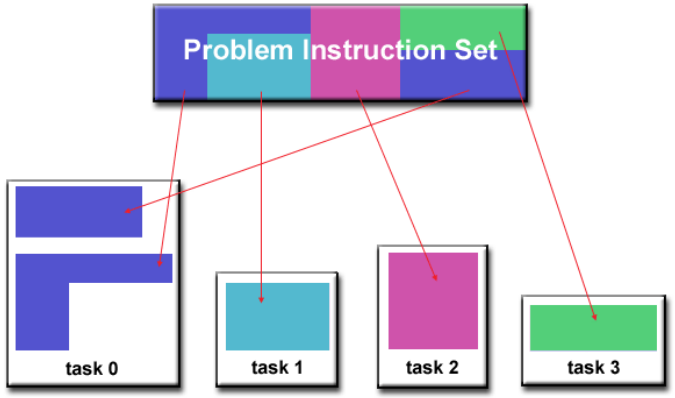
# TASK PARALLELISM

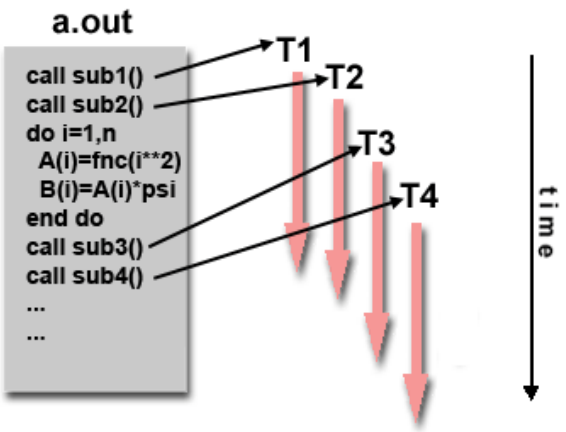
A task is an abstraction of some work to be performed (function, sequence of instructions, etc.)

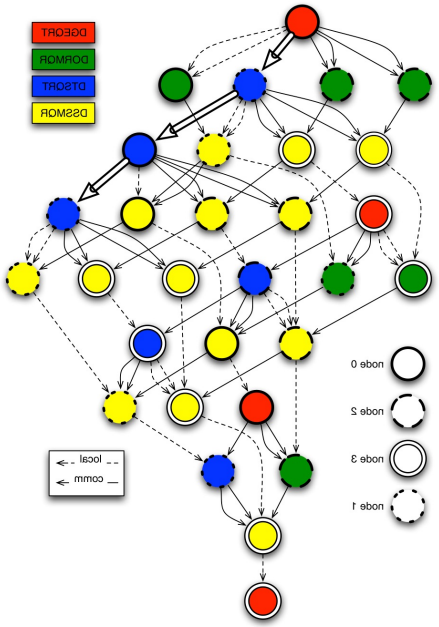
Task parallelism aims at running multiple independent tasks in parallel on different threads/processes/processors/cores

Usually based on functional decomposition of code and thus also referred to as functional parallelism

Generated by user/compiler and managed by compiler/runtime/hardware



# DATA VS. TASK PARALLELISM

**There is often no clear separation between data parallelism and task parallelism**

**Combining these two types of problem decomposition is common and natural**

**Data and task parallelism can both be realized within a shared-memory or a distributed-memory context**

# MAJOR PARALLEL PROGRAMMING APIs

**Explicit Multithreading (eg. C++/Java Threads)**

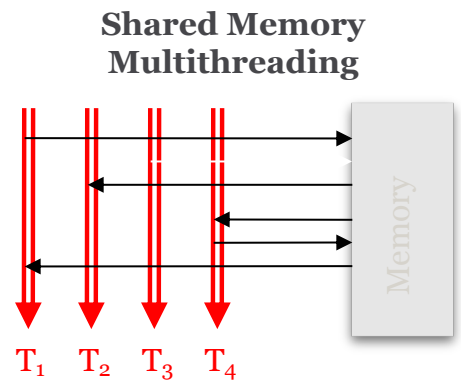
- User manages all details (threads, synchronization,…)

**High(er) Level Parallel Languages/APIs**

- Java Fork-Join Framework, Completable Futures, Streams, …
- Threading Building Blocks (TBB)

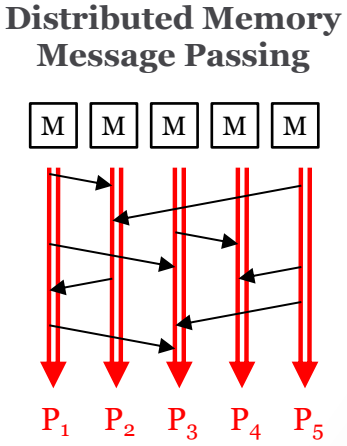
**OpenMP**

- Sequential program + pragmas to mark parallel regions (e.g., loops) and to specify work distribution (schedule)
- Compiler generates multithreaded parallel code




**MPI (Message Passing Interface)**

- User specifies data & work distribution
- User specifies communication/synchronization by calling MPI communication library-routines
- Distributed-memory (clusters) and shared-memory machines



# OPENMP :: 1-D STENCIL EXAMPLE

```
double t[N], t_old[N];
...

for (iter=0; iter < NITER; iter++) {
   #pragma omp parallel for
   for (i=1; i < N-1; i++)
      t[i] = (t_old[i-1] + t_old[i] + t_old[i+1])/3;
   ...
}
```

# JAVA :: 1-D STENCIL EXAMPLE

```
...
double a[] = new double[N+1];
double anew[] = new double[N+1];

...

for(int step=1; step<=MAX_STEPS; step++) {
   IntStream.range(1, N)
            .parallel()
            .forEach(i -> anew[i] = (a[i-1]+a[i]+a[i+1])/3);
   ...
}
```

# MPI :: 1-D STENCIL EXAMPLE

```
N = ...       // original size or arrays
MY_N = N/4+2 // assume 4 processors; need 2 more elements for non-local data
double t[MY_N], t_old[MY_N];   // “allocate” local part only
...
MPI_Comm_size(MPI_COMM_WORLD, &size); //number of ranks (processors)
MPI_Comm_rank(MPI_COMM_WORLD, &rank); //my rank (my id) 0...size-1
...
my_low = (rank == 0) ? 2 : 1;
my_high = (rank == size-1) ? MY_N-1 : MY_N;

for(iter = 0; iter < NITER; iter++) {
  ...
  for(i = my_low; i < my_high-1; i++)
      t[i] = (t_old[i-1] + t_old[i] + t_old[i+1])/3
  ...
  if (rank != 0)        // Receive left boundary element from left neighbor
      MPI_Irecv(&t[0],1,MPI_DOUBLE,rank-1,RIGHT,MPI_COMM_WORLD,&req1);
  if (rank != size-1)  // Receive right boundary from right neighbor
      MPI_Irecv(&t[MY_N-1],1,MPI_DOUBLE,rank+1,LEFT,MPI_COMM_WORLD,&req2);
  if (rank < size-1)   // Send right boundary element to right neighbor
      MPI_Send(&t[MY_N-2],1,MPI_DOUBLE,rank+1,RIGHT,MPI_COMM_WORLD);
  if (rank != 0)        // Send left boundary element to left neighbor
      MPI_Send(&t[1],1,MPI_DOUBLE,rank-1,LEFT,MPI_COMM_WORLD);
  if (rank != 0) MPI_Wait(&req1,&status);
  if (rank != size-1) MPI_Wait(&req2,&status);
...
```

# PARALLEL PATTERNS

**Master-workers**

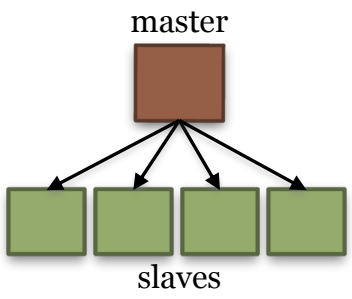
**Producer-consumer flows**

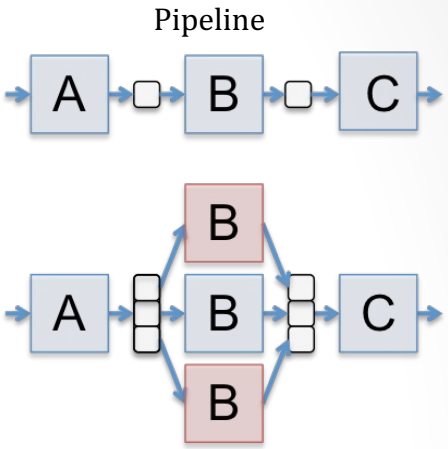
**Shared work queues**

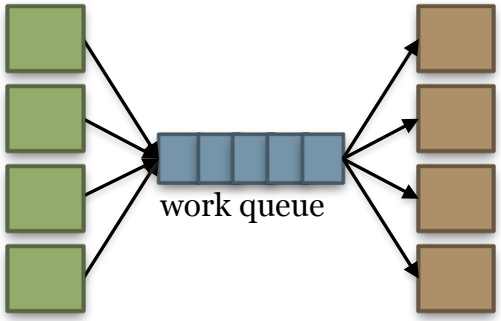
**Map/reduce**

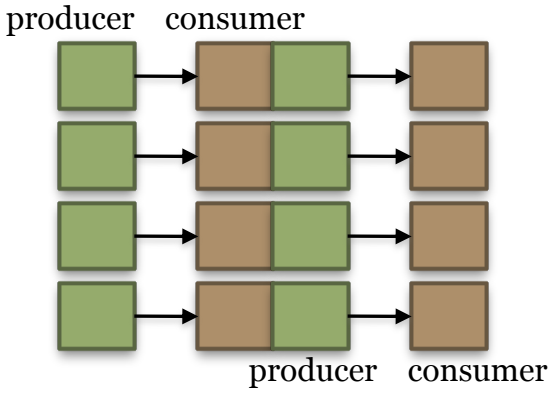
**Pipeline**

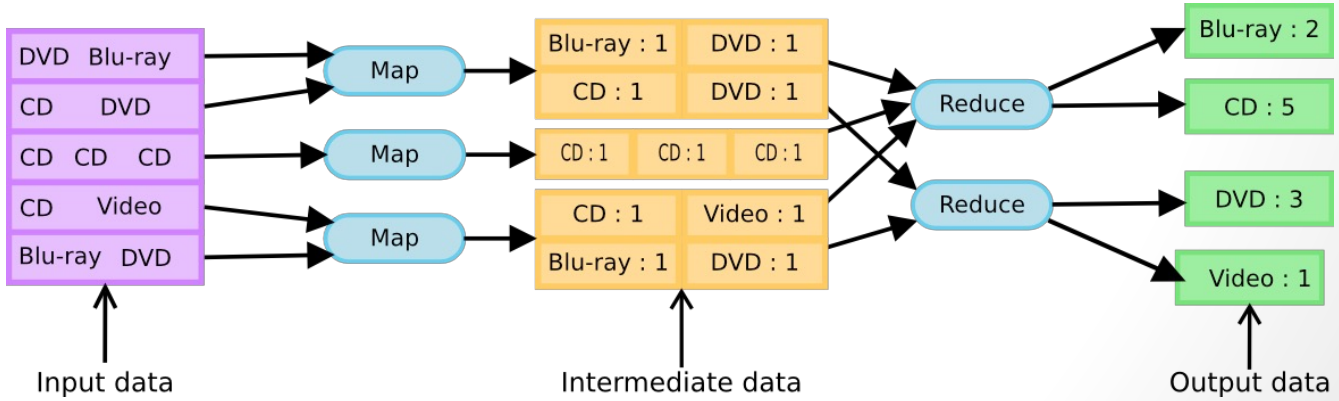
**…**



# DEPENDENCES

**Dependence is a relationship that places constraints on the execution order of computations**

- it determines whether or not computations can be executed in parallel

**There are two types of dependences which must be considered:**

- Control dependence:

```
S1:  if (b ≠ 0)  then
S2:     c = a / b;
```

- Data dependence:

```
S1:  a = …
S2:  c = a / b;
```

# DATA RACES

***Two accesses to a shared variable from different threads form a data race if the accesses are not ordered (synchronized) and at least one access is a write.***

```
int counter;    // shared variable
...
counter++;      // accessed by multiple threads
```

# DEPENDENCE ANALYSIS :: EXAMPLES

**Order of iterations may be semantically irrelevant**

```
for (i=0; i<100; i++) {
   a[i] = a[i] + x;
}
```

```
for (i=1;i<100;i++) {
   a[i] = a[i-1] + x;
}
```

**Order of function calls may be semantically irrelevant**

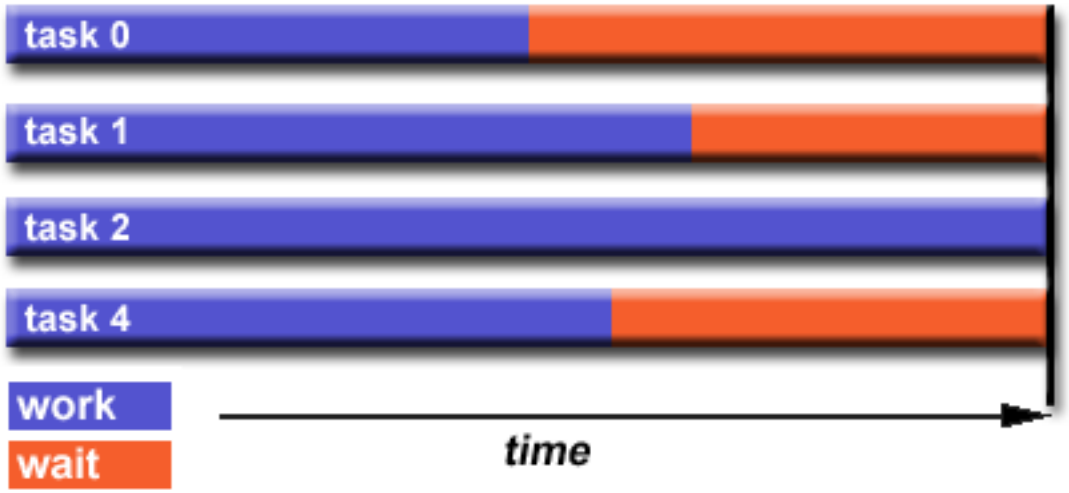
```
func1(a,b);
func2(c,d);
```

```
func1(a,b);
func2(b,c);
```

# LOAD BALANCING

**Practice of distributing approx. equal amounts of work among tasks so that all tasks are kept busy all of the time**

- It can be considered a minimization of task idle time
- Load balancing is important to parallel programs for performance reasons




See https://computing.llnl.gov/tutorials/parallel_comp/

# GRANULARITY

## Computation / Communication Ratio:

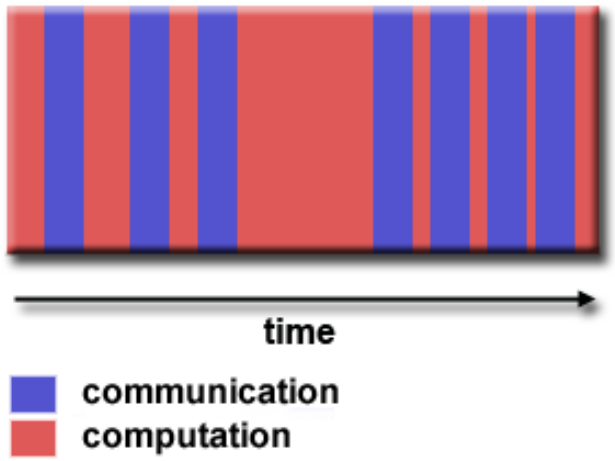
- Periods of computation are typically separated from periods of communication by synchronization events.

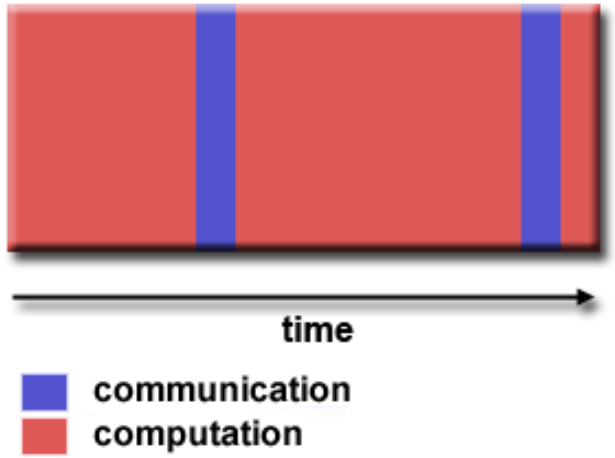
## Fine-grain Parallelism:

- Low computation to communication ratio
- Facilitates load balancing
- Implies high communication overhead and less opportunity for performance enhancement

## Coarse-grain Parallelism:

- High computation to communication ratio
- Implies more opportunity for performance increase
- Harder to load balance efficiently




See https://computing.llnl.gov/tutorials/parallel_comp/

# STRONG VS. WEAK SCALING

**Strong scaling:**

- How does the execution time the solution time vary with the number of processors for a **fixed total problem size**?
- The problem size stays fixed but the number of processors are increased
- **Goal:** find a "sweet spot" that allows the computation to complete in a reasonable amount of time, without too much parallel overhead

# STRONG VS. WEAK SCALING

**Weak scaling:**

- How does the execution  solution time vary with the number of processors for a problem size that increases proportionally to the number of processors (i.e., fixed problem size per processor)
- In this case the problem size (workload) assigned to each processor element stays constant and additional processors are used to solve a larger total problem (one that wouldn't fit on a single node)
- This type of measurement is justification for programs that take a lot of memory (something that is memory-bound)
- Linear (perfect) weak scaling is achieved if the run time stays constant while the workload is increased in direct proportion to the number of processors

# PEAK PERFORMANCE VS. SUSTAINED PERFORMANCE

**Peak Performance = #cores * Flop/cycle * clock frequency measured in Flop/s**

- FLOPs - Floating Point Operations per Second

**Example**

- Intel Xeon Phi (KNL):
  72 * (4*16) * 1,5 GHz → 6.9*1012 Flop/s → 6.9 TFlop/s (SP)

$$\textit{Sustained Performance} = \frac{\textit{number of floating-point operations performed by a program}}{\textit{runtime of program in secs}}$$

# EFFICIENCY = SUSTAINED PERF. / PEAK PERF.

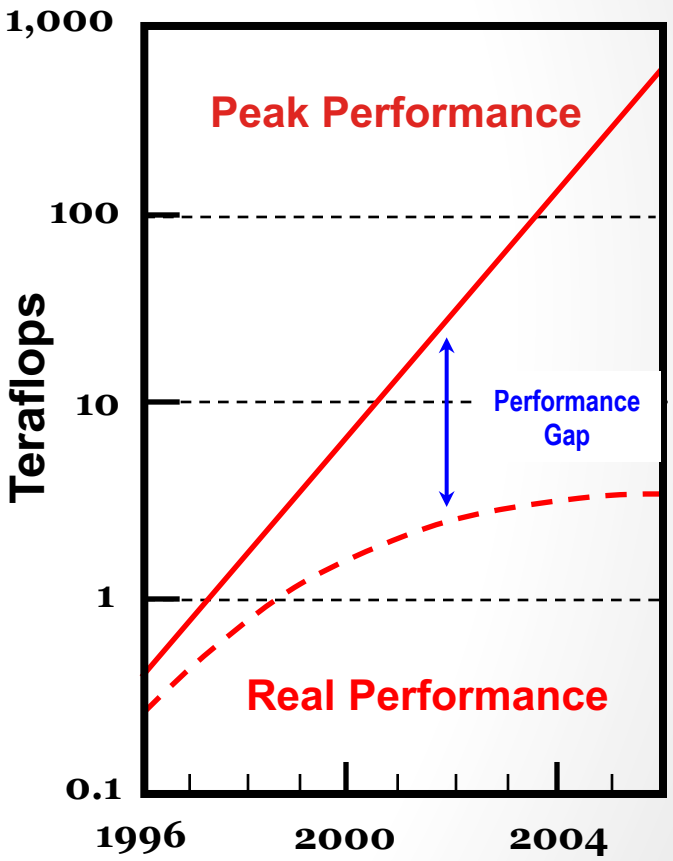
**Efficiency: sustained performance relative to hardware peak performance**

**Efficiency (for real applications) is declining**

- 40-50% on vector supercomputers of 1990s
- Now as little as 5-10% on supercomputers

**Close the gap through ...**

- Algorithms that achieve high performance on a single processor and scale to thousands of processors
- More efficient programming models and tools for massively parallel supercomputers



# SUSTAINED PERFORMANCE HPCG RESULTS, 7/2025

| Rank | Site | Computer | Cores | HPL Rmax (Pflop/s) | TOP500 Rank | HPCG (Pflop/s) | Fraction of Peak |
|---|---|---|---|---|---|---|---|
| 1 | DOE/NNSA/LLNL **United States** | **El Capitan** — AMD 4th Gen EPYC 24C 1.8GHz, Slingshot-11, AMD Instinct MI300A | 11,039,616 | 1742.00 | 1 | 17.41 | 0.6% |
| 2 | RIKEN Center for Computational Science **Japan** | **Supercomputer Fugaku** — A64FX 48C 2.2GHz, Tofu interconnect D | 7,630,848 | 442.01 | 7 | 16.00 | 3.0% |
| 3 | DOE/SC/Oak Ridge National Laboratory **United States** | **Frontier** — AMD Optimized 3rd Generation EPYC 64C 2GHz, Slingshot-11, AMD Instinct MI250X | 9,066,176 | 1353.00 | 2 | 14.05 | 0.7% |
| 4 | DOE/SC/Argonne National Laboratory **United States** | **Aurora** — Xeon CPU Max 9470 52C 2.4GHz, Slingshot-11, Intel Data Center GPU Max | 9,264,128 | 1012.00 | 3 | 5.613 | 0.3% |
| 5 | EuroHPC/CSC **Finland** | **LUMI** — AMD Optimized 3rd Generation EPYC 64C 2GHz, Slingshot-11, AMD Instinct MI250X | 2,752,704 | 379.70 | 9 | 4.587 | 0.9% |
| 6 | Swiss National Supercomputing Centre (CSCS) **Switzerland** | **Alps** — NVIDIA Grace 72C 3.1GHz, Slingshot-11, NVIDIA GH200 Superchip | 2,121,600 | 434.90 | 8 | 3.671 | 0.6% |
| 7 | EuroHPC/CINECA **Italy** | **Leonardo** — Xeon Platinum 8358 32C 2.6GHz, Quad-rail NVIDIA HDR100 Infiniband, NVIDIA A100 SXM4 64 GB | 1,824,768 | 241.20 | 10 | 3.114 | 1.0% |

source: https://www.hpcg-benchmark.org/custom/isc23.html

# THE LAST SLIDE

**Questions?**